

XSS Attack & Defense Lab: Identifying, Exploiting, and Mitigating Cross-Site Scripting Vulnerabilities

Presented By
Lamiya

Presented TO
MD. Nessar Rahman Porag

Objectives

- To understand the fundamentals of Cross-Site Scripting (XSS) vulnerabilities.
- To identify XSS weaknesses in a vulnerable web application.
- To practically exploit Stored, Reflected, and DOM-Based XSS using DVWA.
- To analyze HTTP requests and responses through Burp Suite.
- To apply effective security mechanisms to mitigate XSS attacks.

Lab Environment & Tools

Attacker Machine: Kali Linux

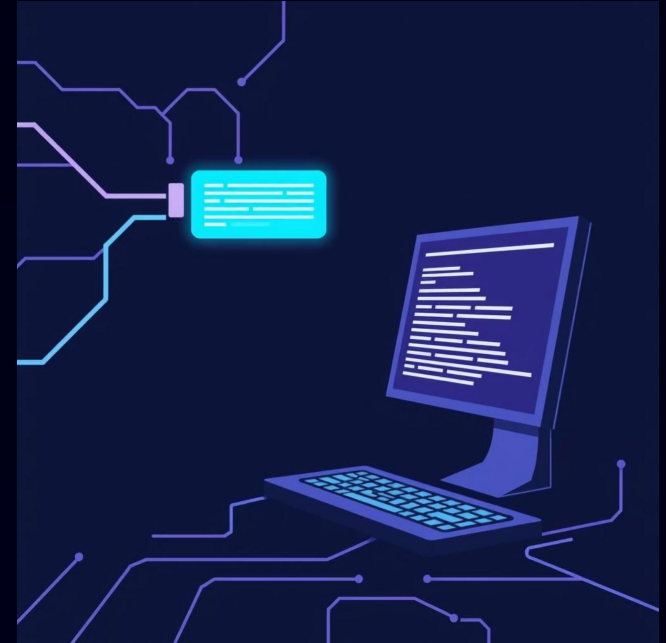
Target Server: Metasploitable2

Vulnerable Application: Damn Vulnerable Web Application (DVWA)

Tools Used: Firefox Browser, Burp Suite, Apache Server, ModSecurity WAF

What is Cross-Site Scripting (XSS)?

- Cross-Site Scripting (XSS) is a common web application security vulnerability
- It allows attackers to inject malicious JavaScript code into a web application
- The injected script executes in the victim's browser
- This can lead to cookie theft, session hijacking, and unauthorized actions



The Attack Surface: Three Primary Types

Understanding the delivery method is key to identifying where your application is most vulnerable.



Reflected XSS

- Script is reflected in server response
- Executes when victim clicks a malicious link



Stored (Persistent) XSS

- Malicious script is stored in the database
- Executes automatically when page loads
- Affects multiple users



DOM-Based XSS

- Occurs entirely on the client side
- Vulnerability exists in JavaScript handling of user input
- Server-side processing is not involved

DVWA Overview

Damn Vulnerable Web Application (DVWA) is a purposely insecure web application. It is designed for learning and practicing web security attacks. The security level was set to Low to demonstrate XSS vulnerabilities.



Fig -1 : Home page



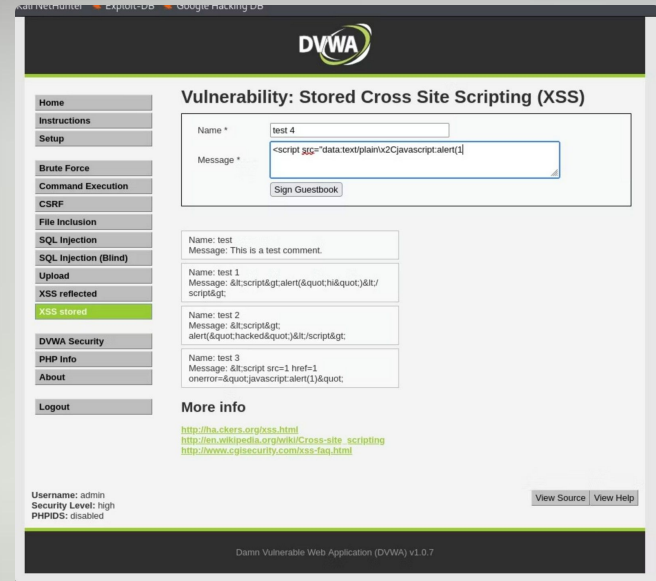
Fig-2 : Security page

XSS Vulnerability Identification

- Application input fields were tested using JavaScript payloads.
- User input was reflected without proper validation or sanitization Multiple XSS vulnerabilities were successfully identified

Stored XSS Exploitation

- A malicious JavaScript payload was submitted through an input form
- The payload was stored in the application database
- The script executed automatically whenever the page was loaded



[Home](#)[Instructions](#)[Setup](#)[Brute Force](#)[Command Execution](#)[CSRF](#)[File Inclusion](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Upload](#)[XSS reflected](#)[XSS stored](#)[DVWA Security](#)[PHP Info](#)[About](#)[Logout](#)

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

Message *

Name: test
Message: This is a test comment.

Name: test 1
Message: <script>alert("hi")</script>

Name: test 2
Message: <script>alert("hacked")</script>

Name: test 3
Message: <script src=1 href=1 onerror="javascript:alert(1)"

Name: test 4
Message: <script src="data:text/plainx2Cjavascript:alert(1

More info

<http://hackers.org/xss.html>
http://en.wikipedia.org/wiki/Cross-site_scripting
<http://www.cgisecurity.com/xss-faq.html>

Username: admin
Security Level: high
PHPIDS: disabled

[View Source](#)[View Help](#)

Reflected XSS Exploitation

- A crafted payload was sent through an HTTP request
- The server reflected the input in its response JavaScript executed immediately in the browser

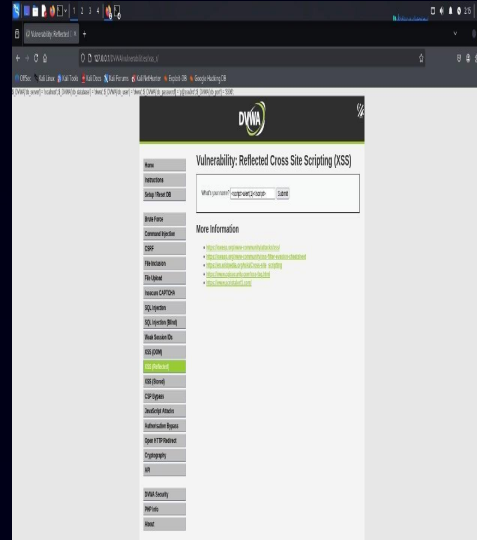


Fig-1 : Input

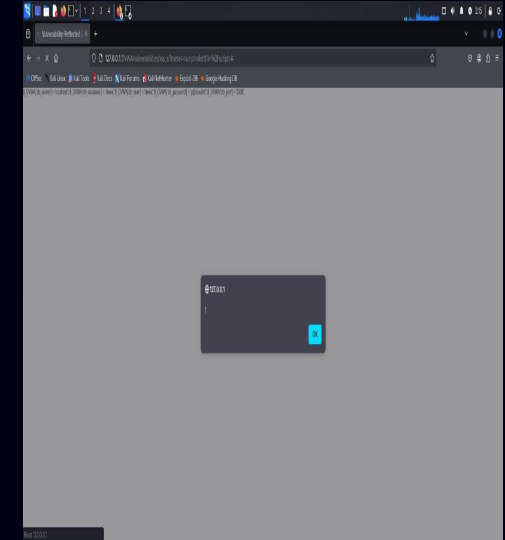
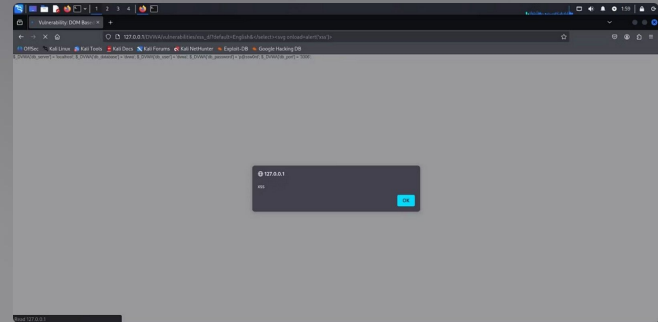


Fig-2 : Output

DOM-Based XSS Exploitation

- Payloads were injected through URL parameters
- Client-side JavaScript processed the malicious input
Script execution occurred without server involvement



Request & Response Analysis Using Burp Suite

- Burp Suite Proxy was enabled to intercept HTTP traffic
- Requests containing malicious payloads were captured
- Server responses confirmed payload reflection or storage

Importance of Burp Suite

- Provides visibility into HTTP request and response flow
- Helps trace how malicious payloads travel through the application
- Acts as strong evidence of XSS vulnerabilities

XSS Stored

Burp Suite Community Edition v2025.11.4 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

Intercept HTTP history Websockets history Match and replace Proxy settings

Filter settings: Hiding CSS and image content; hiding specific extensions

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start respons...
1	https://www.google.com	GET	/warmup.html			200	1366	HTML	html	Warmup Page		✓	172.217.24.68	__Secure-STRP=AD...	10:08:27 22 D...	8080	158
2	https://www.google.com	GET	/warmup.html			200	1366	HTML	html	Warmup Page		✓	172.217.24.68	__Secure-STRP=AD...	10:08:50 22 D...	8080	157
3	http://192.168.0.104	GET	/dwa/security.php			200	4452	HTML	php	Damn Vulnerable Web Ap...			192.168.0.104		10:40:15 22 D...	8080	18
4	http://192.168.0.104	GET	/dwa/vulnerabilities/xss_s/			200	5935	HTML		Damn Vulnerable Web Ap...			192.168.0.104		10:40:17 22 D...	8080	18
5	http://192.168.0.104	POST	/dwa/vulnerabilities/xss_s/		✓	200	6031	HTML		Damn Vulnerable Web Ap...			192.168.0.104		10:41:55 22 D...	8080	13

Inspector

Request attributes 2

Protocol HTTP/1 HTTP/2

Name	Value
Method	POST
Path	/dwa/vulnerabilities/...

Request body parameters 3

Name	Value
txtName	test 6
mtaMessage	<script>alert(2)</scri...
btnSign	Sign Guestbook

Request cookies 2

Name	Value
security	low
PHPSESSID	da649d40e808ed7b...

Request headers 13

Name	Value
Host	192.168.0.104
User-Agent	Mozilla/5.0 (X11; Linu...
Accept	text/html,application...
Accept-Language	en-US,en;q=0.5
Accept-Encoding	gzip, deflate, br

Request

Pretty Raw Hex

```
1 POST /dwa/vulnerabilities/xss_s/ HTTP/1.1
2 Host: 192.168.0.104
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 94
9 Origin: http://192.168.0.104
10 Connection: keep-alive
11 Referer: http://192.168.0.104/dwa/vulnerabilities/xss_s/
12 Cookie: security=low; PHPSESSID=da649d40e808ed7b73f07b9ed96c5de
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 txtName=test+6&mtaMessage=%3Cscript%3Ealert%28%29%3C%2Fscript%3E%3C%0A%0A&btnSign=Sign+Guestbook
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Date: Mon, 22 Dec 2025 16:41:55 GMT
3 Server: Apache/2.2.8 (Ubuntu) DAV/2
4 X-Powered-By: PHP/5.2.4-2ubuntu5.10
5 Pragma: no-cache
6 Cache-Control: no-cache, must-revalidate
7 Expires: Tue, 23 Jun 2009 12:00:00 GMT
8 Content-Length: 5683
9 Keep-Alive: timeout=15, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=utf-8
12
13 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
14 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
15
16 <html xmlns="http://www.w3.org/1999/xhtml">
17
18 <head>
19 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
20
21 <title>
22 Damn Vulnerable Web App (DWA) v1.0.7 :: Vulnerability: Stored Cross Site Scripting
23 (XSS)
24 </title>
25
26 <link rel="stylesheet" type="text/css" href="../../dwa/css/main.css" />
27
28 <link rel="icon" type="image/ico" href="../../favicon.ico" />
29
30 <script type="text/javascript" src="../../dwa/js/dwaPage.js">
```

XSS Reflected

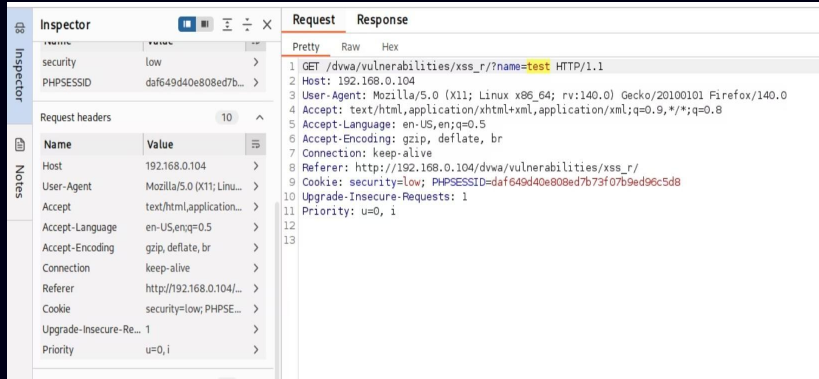


Fig-1: Request

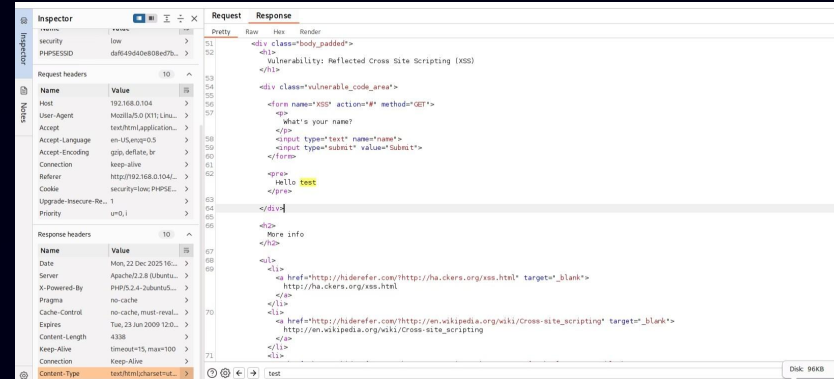


Fig-2: Respons

XSS Dom

Burp Suite Community Edition v2025.7.4 - Temporary Project

Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Logger Organizer

Intercept HTTP history WebSockets history Match and replace Proxy settings

Filter settings: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes
2	https://contile.services.moz...	GET	/M/tiles			204	181				
7	https://contile.services.moz...	GET	/M/tiles			204	181				
6	http://detectportal.firefox.c...	GET	/success.txt?pv6		✓	200	216	text	txt		
5	http://detectportal.firefox.c...	GET	/success.txt?pv6		✓	200	216	text	txt		
3	https://spocs.getpocket.com	POST	/spocs		✓	200	1395	JSON			

Request

1 POST /spocs HTTP/1.1
2 Host: spocs.getpocket.com
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/json
8 Content-Length: 197
9 Priority: u=4
10 Te: trailers
11 Connection: keep-alive

Response

1 HTTP/2 200 OK
2 Content-Type: application/json
3 Strict-Transport-Security: max-age=31536000
4 Date: Tue, 23 Dec 2025 07:24:32 GMT
5 Content-Length: 1168
6 Via: 1.1 google
7 Alt-Svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
8
9 {
10 "settings": {
11 "featureFlags": {
12 "collections": false,
13 "spoc_v2": true
14 },
15 "spocsPerNewTabs": 1,
16 "domainAffinityParameterSets": {
17 "default": {
18 "combinedDomainFactor": 0.5,
19 "frequencyFactor": 0.5,
20 "itemScoreFactor": 1,
21 "multiDomainBoost": 0,
22 "perfectCombinedDomainScore": 2,
23 "perfectFrequencyVisits": 10,
24 "recencyFactor": 0.5
25 },
26 "fully-personalized": {
27 "combinedDomainFactor": 0.5,
28 "frequencyFactor": 0.5,
29 "itemScoreFactor": 0.01,
30 "multiDomainBoost": 0,
31 "perfectCombinedDomainScore": 2,
32 "perfectFrequencyVisits": 10,
33 "recencyFactor": 0.5
34 }
35 }
36 }
37 }

Vulnerability: DOM Based XSS

127.0.0.1/DVWA/vulnerabilities/xss_d?default=<script>al

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB

\$ DVWA[db_server]='localhost'; \$ DVWA[db_database]='dvwa'; \$ DVWA[db_user]='dvwa'; \$ DVWA[db_password]='p@ssw0rd'; \$ DVWA[db_port]='3306';

Home Instructions Setup / Reset DB Brute Force Command Injection CSPC

Weak Session IDs XSS (DOM) XSS (Reflected) XSS (Stored) CSP Bypass JavaScript Attacks Authorisation Bypass Open HTTP Redirect Cryptography API

127.0.0.1 XSS OK

Read 127.0.0.1

Defense Phase Overview

- Defense mechanisms were applied after successful exploitation
- The goal was to prevent JavaScript

execution Multiple security layers were implemented



Stored XSS

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout



Vulnerability: Stored Cross Site Scripting (XSS)

Name *

Message *

Sign Guestbook

Clear Guestbook

Name: test

Message: This is a test comment.

Name: test 1

Message: <script>alert(1)</script>

More Information

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Username: admin

Security Level: Security Level: impossible

Logout

View Source

Reflected XSS



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Hello <script>alert(1)</script>

More Information

- <https://owasp.org/www-community/attacks/xss/>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Username: admin

Security Level: Security Level: impossible

Locale: en

SQLI DB: mysql

Damn Vulnerable Web Application (DVWA)

DOM-Based XSS



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Vulnerability: DOM Based Cross Site Scripting (XSS)

Please choose a language:

English

Select

More Information

- <https://owasp.org/www-community/attacks/xss/>
- https://owasp.org/www-community/attacks/DOM_Based_XSS
- <https://www.acunetix.com/blog/articles/dom-xss-explained/>

View Source

View Help

Username: admin
Security Level: Security Level: impossible
Locale: en
SQLI DB: mysql

Damn Vulnerable Web Application (DVWA)

Defense & Mitigation Strategies

Input Validation

User inputs were validated before processing. Suspicious script tags were filtered. Reduced the risk of malicious injection.



Output Encoding

Special characters were converted into HTML entities. Scripts were displayed as plain text. Browser execution was successfully prevented



ModSecurity Web Application Firewall (WAF)

ModSecurity was integrated with the Apache web server. OWASP Core Rule Set (CRS) was enabled. Incoming requests were inspected for attack patterns

ModSecurity Configuration & Log Analysis

- ModSecurity engine was enabled successfully
- Suspicious XSS payloads were detected

OWASP CRS rules were triggered

- Audit logs were generated for analysis



Result Analysis



Stored XSS

- No alert box was executed
- Payload was automatically HTML- encoded
- Stored XSS vulnerability was mitigated



Reflected XSS

- JavaScript execution was blocked
- Payload appeared as harmless text
- WAF successfully detected malicious patterns



DOM-Based XSS

- No JavaScript execution occurred
- Web page loaded normally
- Client-side protection proved effective

Overall Defense Evaluation

The overall defense implementation in this project proved to be effective in mitigating Cross-Site Scripting (XSS) vulnerabilities. After enabling input validation, output encoding, and deploying ModSecurity Web Application Firewall with the OWASP Core Rule Set, all previously exploited XSS payloads failed to execute. The browser treated injected scripts as harmless text, preventing unauthorized JavaScript execution and protecting user sessions.

Additionally, ModSecurity actively inspected incoming HTTP requests and successfully detected suspicious patterns related to XSS attacks. Relevant security rules were triggered, and detailed audit logs were generated, confirming that the Web Application Firewall was functioning as intended. This layered defense approach significantly reduced the attack surface of the application and demonstrated how combining secure coding practices with automated security controls can effectively protect web applications from XSS attacks.

Conclusion

This project provided a comprehensive understanding of Cross-Site Scripting (XSS) vulnerabilities by combining both theoretical knowledge and practical experimentation. Through the use of Damn Vulnerable Web Application (DVWA), different types of XSS attacks—Stored, Reflected, and DOM-Based—were successfully identified and exploited in a controlled lab environment. These experiments demonstrated how improper input handling and lack of output sanitization can allow malicious JavaScript to execute in a user's browser, leading to serious security risks such as session hijacking and data theft.

Furthermore, the project emphasized the importance of defensive mechanisms in securing web applications. By implementing input validation, output encoding, and deploying ModSecurity with the OWASP Core Rule Set, the effectiveness of layered security defenses was clearly observed. After enabling these protections, malicious scripts failed to execute, and suspicious activities were detected and logged by the Web Application Firewall. Overall, this project highlights the critical need for secure coding practices and proactive security measures to mitigate XSS vulnerabilities and strengthen the security of modern web applications.